Tecnológico de Monterrey

Campus Monterrey

MODULAR SCIENTIFIC COMPUTING PIPELINE FOR TRAFFIC FLOW PREDICTION

Urban Mobility

Applied computing (TC6039 - 1)

Ph.D. Luis Alberto Muñoz Ubando

Team members:

- Rodrigo Gutierrez Garza C1 (A00827978)
- María del Ángel Guerrero Alvarado C2 (A01705412)
- Kenia Gabriela Hermida Nuñéz C3 (A01198167)
- Victor Hugo Gutierrez Nuñéz C4 (A00845799)
- Fernando Arturo Rodríguez Lee C5 (A01114887)
- Edson Fernando Acosta C6-C7 (A01796499)

April 10th, 2026

Contents

Introduction **3**

Motivation . . . 3

Data Origin . . . 3

Scientific Hypothesis . . . 3

System Architecture and Module Descriptions **4**

Module C1: Data Acquisition and Exploratory Data Analysis . . . 5

Explorative Data Analysis . . . 5

Module C2: Regression Analysis . . . 7

Design Matrix and Polynomial Expansion . . . 7

Regression Fitting and Optimization . . . 7

Validation via K-Fold Cross-Validation . . . 7

Residual Analysis and Homoscedasticity . . . 8

Results and Discussion . . . 8

Module C3: Optimization and Sensitivity Analysis . . . 10

Data Preparation . . . 10

Objective Function . . . 11

Optimization Procedure . . . 11

Convergence Analysis . . . 12

Sensitivity Analysis . . . 12

Final Model . . . 13

Module C4: Classical Machine Learning Baseline . . . 14

Overview and Objective . . . 14

Data Preparation and Aggregation . . . 14

Feature Engineering . . . 14

Model Architectures and Hyperparameters . . . 15

Results 16
Feature Importance Analysis 16
Prediction Timeline 17
Contribution to the Project Hypothesis 17
Module C5: Deep Learning Predictor 19
Feature Engineering and Inputs 19
Model Architecture and Internal Calculations 19
Evaluation Metrics and Accuracy 20
Module C6: Evaluation and Visual Analytics 21
Module C7: Orchestration 22

Results **23**
Deep Learning Model Results and Analysis (C5) 23
Learning Curve Analysis 23
Temporal Validation: Real vs. Predicted 23

Conclusions **24**

Introduction

Motivation

Monterrey, Nuevo León, is a rapidly growing metropolitan area that experiences severe vehicular congestion, leading to increased travel times, economic inefficiencies, and higher greenhouse gas emissions. Accurate traffic forecasting is a fundamental component of Intelligent Transportation Systems (ITS) and smart city planning. Developing a reliable model to predict traffic volume can help mitigate congestion, optimize traffic light sequencing, and improve overall urban mobility.

Data Origin

The data utilized in this project consists of vehicular traffic records in the form of events of license plate detection, collected from a specific intersection in Monterrey over a two-month period (January and February 2026). The dataset is structured as a time series, capturing discrete vehicular passages that have been aggregated into hourly volumes. Key independent variables (features) extracted include temporal markers such as the specific hour of the day (0-23) and the day of the week, allowing for the analysis of both daily and weekly seasonality.

Scientific Hypothesis

Vehicular traffic flow is highly cyclical but subject to stochastic noise. Our primary hypothesis is that a modular scientific computing pipeline can accurately model and predict hourly traffic volume by exclusively leveraging temporal features (hour and day). Furthermore, we hypothesize that while Classical Machine Learning algorithms (e.g., Polynomial Regression, Random Forests) can capture basic seasonal trends, a Deep Learning architecture (Multilayer Perceptron) will yield a superior predictive performance by better approximating the non-linear complexities of peak-hour traffic dynamics.

System Architecture and Module Descriptions

To ensure reproducibility, robust testing, and a clear separation of concerns, the computational pipeline is built using a strictly modular architecture. The system orchestrates the flow of data through six core components, taking raw traffic records and transforming them into actionable, visual predictions.

The following are the 7 components of the project:

- **C1** Data Loader
- **C2** Regression Analysis
- **C3** Optimization Algorithms
- **C4** Machine Learning Models
- **C5** Deep Learning
- **C6** Visualization
- **C7** Orchestration

The pipeline flows from one component to the subsequent, based on the data that is provided by component C1. In parallel, components C2, C4, and C5 fit the corresponding model to predict the hourly traffic flow. The component C3 serves as optimization of the results in component C2. Finally, the component C6 serves for visualization of the results from the previous components and C7 the orchestration for the execution of the complete pipeline in a complete 'clean' computer for terms of reproducibility.

Module C1: Data Acquisition and Exploratory Data Analysis

This initial module handles the ingestion of the raw vehicular traffic dataset (`df.csv`). It is responsible for establishing a stable data pipeline that reads the records, parses timestamp strings into standard `datetime` objects, verifies data types, and structures the information into a tabular format using the `pandas` library. The module ensures that the time-series data is accurately indexed for downstream processing.

Before predictive modeling occurs, this component ensures data integrity and extracts initial statistical insights. It algorithmically scans the dataset to identify and handle missing values, duplicate entries, and anomalous traffic spikes (outliers). Additionally, it performs preliminary statistical profiling to uncover the baseline distributions of vehicular flow across different times of the day and days of the week, establishing the contextual foundation for the predictive models. This component contains the following programs:

- data_loader.py

  -Reads raw data, cleans and reports preliminary statistics about the dataframe.

- test_data_loader.py

  -Declares and test normal and edge functionality cases.

The data_loader.py code contains the following 3 functions: *load_data(path)* that receives a path (./data/raw by default) and generates a pandas dataframe with the data. Then, *clean_data(df)* receives the dataframe and cleans it by handling NaN values, corrupted data and assures it is within the realistic bounds (e.g. 0-23 hours per day). The last function *eda_summary.py* delivers a dictionary with descriptive statistics and exploratory figures.

Explorative Data Analysis

As shown in Figure 1, the traffic flow has not a linear behaviour but it experience major tendencies in the morning (6-9 am) and after 12 pm. Figure 2 shows the minute histogram, which only reveals major flow at the end of each hour. The gaps may be explained by the semaphoric

cycles (whenever the traffic light is red, there is no flow reported). The weekday number histogram (see Figure 3) reveals an increasing flow from monday to wednesday and a decrement on thursday and friday. The correlation matrix may be inspected in Figure 4, while the pairplot matrix is available in Figure 5.

Additional analysis details are available in the file *descriptive_statistics.json* under the folder ./report/figures.

Module C2: Regression Analysis

This section describes the implementation of the **Polynomial Regression** module (C2), which utilizes Ordinary Least Squares (OLS) to model relationships between vehicular variables. Unlike the neural network approach, this module focuses on statistical interpretability and polynomial complexity control.

Design Matrix and Polynomial Expansion

The model handles both simple linear and multivariate predictors. To capture non-linear trends, the system implements a *Polynomial Design Matrix* builder. Given an input feature x, the model constructs a matrix $\mathbf{X}$ by expanding the features up to a degree $d \in \{1, 2, 3\}$ and prepending an intercept term (bias):

$$\mathbf{X} = [1, x, x^2, \ldots, x^d] \tag{1}$$

This allows the linear solver to fit non-linear curves (quadratic or cubic) within a linear regression framework.

Regression Fitting and Optimization

The parameters (coefficients β) are estimated by solving the normal equations via `numpy.linalg.lstsq`, which minimizes the sum of squared residuals. The prediction is calculated as:

$$\hat{y} = \mathbf{X}\beta \tag{2}$$

The module also computes the **Coefficient of Determination (R^2)**, which measures the proportion of variance in the dependent variable that is predictable from the independent variables:

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} \tag{3}$$

Validation via K-Fold Cross-Validation

To ensure the model generalizes well and to select the optimal polynomial degree, a **K-Fold Cross-Validation** ($k = 5$) procedure is implemented. The dataset is partitioned into k subsets; the

model is trained on $k-1$ folds and validated on the remaining fold. This process is repeated k times to provide:

- **Mean RMSE:** The average error across all validation folds.
- **Std RMSE:** The stability of the model (standard deviation of the error).

Residual Analysis and Homoscedasticity

A critical component of this module is the diagnostic plotting of residuals ($e_i = y_i - \hat{y}_i$). The implementation includes:

- **Residuals vs. Predicted Plot:** Used to detect heteroscedasticity. It includes binned standard deviation bands ($\pm 1\sigma$ and $\pm 2\sigma$) to visualize if error variance changes across the prediction range.
- **Normality Check:** A histogram of residuals overlaid with a theoretical normal distribution curve to verify the OLS assumption that errors are normally distributed.

Results and Discussion

I tested the module using vehicular flow data aggregated by hour and weekday, which produced 168 observations (24 hours $\times$ 7 weekdays). The features were `hour` and `weekday_number`, and the target variable was the vehicle count per combination. I fitted three polynomial models (degrees 1, 2, and 3) and evaluated them using 5-fold cross-validation.

Table 1 summarizes the training and cross-validation metrics for each model.

The cubic model achieved the best performance, with $R^2_{CV} = 0.5437$ and $\text{RMSE}_{CV} = 252.28$. Increasing the polynomial degree improved R^2 numerically (from 0.26 to 0.58), but the residual structure remained similar across all three models, as shown in Figure 6, which suggested that the issue was not the polynomial degree itself.

The closeness between R^2_{train} and R^2_{CV} in all three models confirmed that there was no overfitting. Table 2 presents the per-fold metrics for the cubic model, where the variability across

folds was moderate (R^2 ranged from 0.36 to 0.73). I attributed this variability to the limited dataset size, since each fold contained approximately 34 observations.

The residual analysis shown in Figure 7 revealed two important diagnostic issues. First, the binned standard deviation bands clearly widened as the predicted values increased, which indicated heteroscedasticity: the model produced larger errors during high traffic hours than during low traffic hours. Second, the histogram of residuals showed a positive skew with a long right tail, which violated the OLS assumption of normally distributed residuals. I also observed that the cubic model produced negative predictions for some inputs, which had no physical meaning since vehicle counts cannot be negative.

These findings led me to conclude that the polynomial regression approach, while methodologically correct and useful as a baseline, was not the ideal model for this particular dataset. The variables `hour` and `weekday_number` are cyclic and categorical in nature rather than continuous, since hour 23 and hour 0 are adjacent in time but maximally distant on a numerical scale. A polynomial curve cannot represent this circular structure, nor can it capture the dual peak pattern typical of vehicular traffic (morning and evening rush hours separated by a midday valley). The polynomial regression therefore established a baseline performance, and I expect that the machine learning and deep learning components (C4 and C5) will achieve better results by handling these variables more appropriately.

Module C3: Optimization and Sensitivity Analysis

This module focuses on optimizing the regression coefficients obtained from Module C2 in order to minimize the prediction error. The optimization is performed by directly minimizing the Root Mean Squared Error (RMSE) between the predicted and observed traffic values, measured in vehicles per hour.

The optimization procedure combines two complementary approaches: a custom implementation of Gradient Descent and a quasi-Newton method (BFGS) provided by `scipy.optimize`. The goal is to refine the regression parameters obtained in the previous module and verify that the solution corresponds to a stable minimum.

Data Preparation

The inputs to this module are the feature matrix $X \in \mathbb{R}^{n\times 2}$ and the target vector $y \in \mathbb{R}^n$, where y represents traffic flow in vehicles per hour. The two input features are the weekday index (x_1) and the hour of day (x_2).

If the input data is not already normalized, the code applies standardization using:

$$X_{\text{norm}} = \frac{X - \mu}{\sigma} \tag{4}$$

where μ and σ correspond to the mean and standard deviation of each feature. To prevent numerical instability, very small standard deviation values are replaced by 1 before normalization. If the data is already normalized (controlled by the parameter `pre_normalized=True`), this step is skipped to avoid double scaling.

After normalization, a polynomial design matrix of degree $d = 2$ is constructed using the function `_build_design_matrix`. This expands the feature space as

$$\mathbf{\Phi} = \left[\, \mathbf{1} \;\middle|\; \tilde{x}_1 \;\middle|\; \tilde{x}_2 \;\middle|\; \tilde{x}_1^2 \;\middle|\; \tilde{x}_2^2 \,\right] \in \mathbb{R}^{n\times 5} \tag{5}$$

where $\tilde{x}_1$ and $\tilde{x}_2$ denote the normalized features. The resulting model therefore contains five parameters.

Objective Function

The optimization objective is defined as the Root Mean Squared Error:

$$f(\boldsymbol{\theta}) = \text{RMSE}(\boldsymbol{\theta}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{6}$$

where the predicted values are obtained through the linear model

$$\hat{\mathbf{y}} = \boldsymbol{\Phi}\boldsymbol{\theta}. \tag{7}$$

The analytical gradient of the RMSE with respect to the parameter vector $\boldsymbol{\theta}$ is given by

$$\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) = -\frac{1}{n \cdot f(\boldsymbol{\theta})}\boldsymbol{\Phi}^\top(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}). \tag{8}$$

This gradient expression allows efficient computation of parameter updates during the optimization process.

Optimization Procedure

The function `optimize_parameters` implements the following steps:

1. Construction of the polynomial design matrix $\boldsymbol{\Phi}$.

2. Initialization of the parameter vector for Gradient Descent:

$$\boldsymbol{\theta}^{(0)} = \mathbf{0} \in \mathbb{R}^5. \tag{9}$$

   Starting from a zero vector allows the convergence behavior of the optimizer to be clearly observed.

3. Gradient Descent updates using the rule

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha \nabla f(\boldsymbol{\theta}_k) \tag{10}$$

where α is the learning rate.

4. Early stopping when the change in objective value satisfies

$$|f(\boldsymbol{\theta}_k) - f(\boldsymbol{\theta}_{k-1})| < 10^{-7}. \tag{11}$$

5. Parallel optimization using the BFGS quasi-Newton algorithm through `scipy.optimize.minimize`.

The final parameter vector is selected as the solution producing the lowest RMSE:

$$\boldsymbol{\theta}^* = \begin{cases} \boldsymbol{\theta}_{\text{BFGS}} & \text{if RMSE}_{\text{BFGS}} \leq \text{RMSE}_{\text{GD}} \\ \boldsymbol{\theta}_{\text{GD}} & \text{otherwise.} \end{cases} \tag{12}$$

Convergence Analysis

Figure 8 illustrates the convergence behavior of the Gradient Descent algorithm.
The optimization begins with a relatively high error and rapidly decreases during the first iterations. The curve gradually stabilizes as the algorithm approaches the optimal solution. Convergence occurs after approximately 4000 iterations, reaching a final RMSE of

$$\text{RMSE} = 53.7856 \text{ vehicles/hour}. \tag{13}$$

The BFGS optimizer achieved the same final RMSE value, indicating that both methods converged to the same minimum of the objective function. Because of its faster convergence properties in general optimization problems, the BFGS solution was selected as the final parameter vector.

Sensitivity Analysis

To evaluate the stability of the optimized model, a sensitivity analysis was performed by perturbing the parameter θ_1 while keeping the other coefficients fixed. The perturbed parameter is defined as

$$\theta_1^{(\text{perturbed})} = \theta_1 \cdot m, \quad m \in [0.5, 1.5]. \tag{14}$$

For each perturbed value, the RMSE is recomputed. The results are shown in Figure 9.

The resulting curve exhibits a convex, U-shaped profile, confirming that the optimized parameter corresponds to a true minimum of the objective function. Small deviations from the optimal value increase the RMSE, indicating that the solution is locally stable.

The optimal parameter value identified during this analysis is

$$\theta_1^* \approx -22.75. \tag{15}$$

Final Model

The optimized parameter vector obtained after the optimization process is

$$\boldsymbol{\theta}^* = [175.36,\ -22.75,\ 23.50,\ -8.56,\ -59.27]\,. \tag{16}$$

Substituting these parameters into the regression model yields the final predictive equation:

$$\hat{y} = 175.36 - 22.75\,\tilde{x}_1 + 23.50\,\tilde{x}_2 - 8.56\,\tilde{x}_1^2 - 59.27\,\tilde{x}_2^2. \tag{17}$$

This equation captures the nonlinear relationship between traffic flow, weekday index, and time of day. The quadratic terms allow the model to represent variations in traffic intensity across the day, such as morning and evening peaks, while the linear terms capture broader temporal trends.

Overall, the optimization module successfully verifies convergence through iterative optimization and confirms parameter stability through sensitivity analysis.

Module C4: Classical Machine Learning Baseline

Overview and Objective

Module C4 implements and compares three classical supervised learning models to predict hourly vehicular traffic flow at the Nuevo Sur intersection in Monterrey. The module receives the cleaned, aggregated dataset produced by C1, applies cyclic feature engineering, trains each model, evaluates performance using RMSE and R^2, and generates comparative visualizations. Its primary role within the pipeline is to establish a strong analytical baseline that the Deep Learning module (C5) will subsequently attempt to surpass, directly supporting the project's central hypothesis.

Data Preparation and Aggregation

The raw dataset consists of 74,324 individual vehicle detection events recorded by a License Plate Recognition (LPR) camera between January 18 and March 3, 2026. After filtering weekends, the records were aggregated into hourly buckets by grouping on `(date, hour, weekday_number)`, producing **534 hourly samples**. Each sample represents the total number of vehicles detected in a given hour of a given weekday, which becomes the regression target: `vehicles_per_hour`.

The dataset was split chronologically 80/20 to preserve temporal structure:

- **Training set:** January 19 – February 23 (427 samples)
- **Test set:** February 23 – March 3 (107 samples)

A chronological split was chosen deliberately so that the test set represents a true out-of-sample future prediction scenario, consistent with how traffic forecasting models are evaluated in practice.

Feature Engineering

A key challenge in this dataset is the large difference in average traffic volume between January (mean $\approx$ 265 veh/h at peak) and February–March (mean $\approx$ 100 veh/h at peak), reflecting a genuine seasonal shift in the data collection period. Without encoding this temporal context, models trained on January data extrapolate incorrectly into February and March, yielding R^2 values close to zero.

To address this, all temporal variables were encoded using **cyclic (trigonometric) features** rather than raw integers. Raw integer encoding imposes a false linear ordering: for example, it makes Hour 23 and Hour 0 appear far apart when they are in fact adjacent. Cyclic encoding maps each variable onto the unit circle via sine and cosine, preserving true adjacency and periodicity.

The 8 input features used by all three models are listed in Table 3.

The second harmonic of the hour (`hour_sin2`, `hour_cos2`) was included to capture the double-peak structure of weekday traffic: a morning rush around 7–8 h and an evening accumulation around 17–18 h, which a single sinusoid cannot represent. The month cyclic pair (`month_sin`, `month_cos`) proved to be the most impactful addition: including it raised the Decision Tree R^2 from 0.10 to 0.93 by allowing the models to distinguish the high-traffic January period from the lower-volume February–March window.

All features were standardized using `StandardScaler` inside individual `sklearn` `Pipeline` objects before model fitting, ensuring a fair comparison across models with different sensitivity to feature scale.

Model Architectures and Hyperparameters

Three models were trained and compared:

Linear Regression. Ordinary Least Squares (OLS) regression was implemented using `sklearn`'s `LinearRegression`, which internally solves the normal equations via `numpy.linalg.lstsq`. This model has no tunable hyperparameters and serves as the interpretable baseline. Its fitted intercept of 130.42 veh/h represents the expected hourly volume when all cyclic features equal zero. Algorithmic complexity: $\mathcal{O}(nm^2)$ where n is the number of samples and m the number of features.

Decision Tree Regressor. A single regression tree with `max_depth = 5` was trained, producing 61 internal nodes and 31 leaf nodes. The root split was on `month_cos` ≤ 0.65, which partitions the data into the high-traffic January period versus the lower-traffic February–March window directly from the first split. The tree uses Mean Squared Error (MSE) as the split criterion and `random_state = 42` for reproducibility. Algorithmic complexity: $\mathcal{O}(nm \log n)$.

Random Forest Regressor. An ensemble of 100 independent decision trees, each trained on a bootstrap sample of the data with random feature subsets at each split. Individual trees were grown to `max_depth = 7`. The final prediction is the arithmetic mean of all 100 tree predictions, which reduces variance through ensemble averaging. Parameters: `n_estimators = 100`, `max_depth = 7`, `random_state = 42`, `n_jobs=-1` (parallel training on all CPU cores). Algorithmic complexity: $\mathcal{O}(t \cdot n \cdot m \cdot \log n)$ where $t = 100$ is the number of trees.

Performance was evaluated on the held-out test set using RMSE and R^2 — consistent with the metrics used in C2 and C3 — and additionally validated with 5-fold cross-validation on the training set to assess generalization stability.

Results

Table 4 summarizes the performance of the three models on the test set.

The Random Forest achieved the best test performance with $RMSE_{test}$ = 10.99 veh/h and R^2_{test} = 0.939, meaning the model explains 93.9% of the variance in the hourly vehicle count during the test period. The Decision Tree achieved comparable accuracy (R^2 = 0.930) with a simpler, fully interpretable structure.

Linear Regression underperforms significantly (R^2 = 0.276), which was expected: the relationship between cyclic time features and traffic volume is inherently non-linear, and a linear model cannot capture the sharp morning peak at 7–8 h or the plateau behavior during midday hours, as shown in Figure 11.

The high cross-validation RMSE values relative to the test RMSE reflect the temporal heterogeneity of the dataset: January folds (high traffic) are structurally different from February–March folds (lower traffic), so cross-validation across these periods introduces higher variance than the chronological test split.

Feature Importance Analysis

Figure 10 shows the feature importance scores extracted from the Decision Tree and Random Forest models.

In both models, `hour_cos` and `month_cos` are the dominant features. `month_cos` being the root split of the Decision Tree (importance ≈ 37%) confirms that the monthly seasonal shift is the primary source of variation in this dataset. `hour_cos` (importance ≈ 36% in the Decision Tree, 35% in the Random Forest) captures the primary daily traffic cycle: low volumes at night and high volumes during the day.

The weekday features (`weekday_sin`, `weekday_cos`) contribute modestly (<10%), suggesting that the day of the week within the Monday–Friday window has a secondary effect compared to the time of day and the month.

Prediction Timeline

Figure 11 shows the actual versus predicted traffic flow for each model during the test period (February 23 – March 3, 2026).

The Random Forest closely tracks the real signal, including low overnight volumes (near zero) and morning peaks. The Decision Tree produces slightly more discrete step-like predictions due to its leaf structure but still achieves high accuracy. Linear Regression captures the general daily cycle shape but systematically over-predicts during low-traffic hours and under-predicts sharp morning peaks, confirming the non-linearity of the problem.

Contribution to the Project Hypothesis

The results from C4 directly support the project's scientific hypothesis, which states that Deep Learning architectures will outperform classical regression and machine learning models by at least 15% in forecasting non-linear peak-hour traffic surges.

The Random Forest establishes a strong baseline of $R^2 = 0.939$ and RMSE = 10.99 veh/h on the February–March test period. For C5 to demonstrate the hypothesized 15% superiority, it must achieve $RMSE_{test} < 9.34$ veh/h under comparable conditions. Crucially, Linear Regression's poor performance ($R^2 = 0.276$) confirms that the traffic signal contains non-linear dynamics that tree-based and neural network architectures can better capture — a finding that reinforces the scientific motivation for C5.

The feature importance analysis also reveals that the problem is fundamentally temporal and

seasonal: the most predictive variables are cyclic representations of the hour and the month. Any architecture in C5 that models temporal sequences explicitly (e.g., LSTM) has an inherent structural advantage over the static feature vectors used in C4, since it can leverage the sequential dependence between consecutive hours rather than treating each hour independently.

Module C5: Deep Learning Predictor

This section details the implementation of a Multilayer Perceptron (MLP) developed in **PyTorch** for vehicular flow prediction. The model processes tabular data transformed to capture hourly and weekly seasonality.

Feature Engineering and Inputs

To ensure the neural network correctly interprets the cyclical nature of time, input variables were transformed using trigonometric functions. The input vector $X \in \mathbb{R}^4$ consists of:

- **Hourly Cycle:** $\sin$ and $\cos$ components of the hour (24-hour period).
- **Weekly Cycle:** $\sin$ and $\cos$ components of the day of the week (7-day period).

This encoding allows the model to understand that values such as 23:00 and 00:00 are temporally contiguous.

Model Architecture and Internal Calculations

The architecture is defined as a deep dense network with the following layers:

1. **Initial Linear Layer:** Projects the 4 inputs into a 64-dimensional hidden space.
2. **Regularization Blocks: Batch Normalization** was implemented to stabilize internal activations, alongside **Dropout** (0.1 rate) to mitigate overfitting.
3. **ReLU Activation:** The Rectified Linear Unit function is used in hidden layers to introduce non-linearity.
4. **Output Layer:** A final neuron with linear activation that estimates the volume of vehicles per hour.

The model utilizes the **Adam** optimizer to adjust weights through backpropagation, minimizing the Mean Squared Error (MSE) loss function.

Evaluation Metrics and Accuracy

Model performance is evaluated by the magnitude of errors produced on the validation set, primarily using:

- **RMSE (Root Mean Squared Error):** Provides a measure of the standard deviation of the residuals. It is calculated as:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{18}$$

- **MAE (Mean Absolute Error):** Represents the average absolute error in terms of vehicle units, being less sensitive to outliers than RMSE:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{19}$$

Finally, a visual comparison between actual time series and model predictions is performed to verify the MLP's ability to track traffic trends during peak and off-peak hours.

Module C6: Evaluation and Visual Analytics

The final component synthesizes the outputs from all previous modules to generate comprehensive quantitative metrics and visual analytics using `matplotlib` and `seaborn`. Rather than solely plotting final predictions, this module visualizes the entire pipeline's effectiveness. It generates distribution histograms and correlation matrices from the EDA phase (C2 and C3), overlays average hourly traffic profiles (e.g., contrasting January versus February), and produces comparative learning curves alongside actual-versus-predicted traffic graphs to evaluate the models generated in C4 and C5.

Module C7: Orchestration

The project's architecture and workflow follow the best practices for scientific computing established by Wilson et al. (2014). The implementation focuses on the following key areas:

- **Automation and Workflow:** A `main.py` script orchestrates the entire pipeline, providing real-time progress updates. A `Makefile` is included to standardize execution via specific targets: `run`, `test`, `clean`, and `report`.
- **Reproducibility:** To ensure environment consistency, a `requirements.txt` file was generated using `pip freeze`, documenting the exact versions of all dependencies.
- **Quality Assurance:** The codebase includes a suite of automated tests using `pytest`. Each module is supported by at least two unit tests covering core logic and edge cases.
- **VCS and Documentation:** The repository is structured for version control with a `.gitignore` file to exclude unnecessary artifacts. Detailed, step-by-step setup and usage instructions are provided in the `README.md` file.

Results

Deep Learning Model Results and Analysis (C5)

After training the MLP model for 100 epochs, its learning capacity and generalization performance were evaluated by analyzing the learning curves and comparing predictions against ground truth values.

Learning Curve Analysis

Figure 12 illustrates the error evolution during the optimization process. Stable convergence is observed in both the training and validation sets.

- **Loss-Curve (RMSE):** The loss function decreases sharply during the first 10 epochs, stabilizing near 120 vehicles of error for training and 150 for validation. The constant gap between both curves suggests a slight bias, but the lack of divergence confirms that no critical overfitting is occurring.
- **MAE vs Epoch:** The absolute error follows a similar trend, achieving an average precision of approximately 70 vehicles per hour on the validation set. Minor oscillations in the validation curve are characteristic of the Adam optimizer as it navigates the loss landscape of noisy traffic data.

Temporal Validation: Real vs. Predicted

To verify the practical utility of the model, predictions were compared against observed vehicular traffic over a 100-hour sequence.

As shown in Figure 13, the model demonstrates a high capability for capturing the **hourly seasonality** of vehicular flow. The MLP correctly identifies low-demand periods (valleys) and significant increases during peak hours.

While there is a slight tendency to overestimate maximum peaks in certain cycles (e.g., near temporal indices 15 and 35), the model closely tracks the overall trend. This validates that the feature engineering process—using sinusoidal functions—effectively allowed the network to learn the temporal structure of the problem.

Conclusions

The development of this modular scientific computing system allowed for a comprehensive analysis of urban mobility, successfully integrating statistical regression techniques (C2) and deep learning architectures (C5). According to the results obtained, the following conclusions are drawn:

- **Model Performance:** The comparison between classical and deep learning models reveals that while polynomial regression (C2) provides a transparent baseline for understanding general trends, the MLP architecture (C5) significantly improves the capture of non-linear temporal patterns[cite: 125, 145]. The inclusion of cyclical feature engineering (sin/cos transformations) was pivotal in reducing error during peak-to-valley transitions.

- **Scientific Rigor and Reproducibility:** Following the best practices for scientific computing by Wilson et al. (2014), the project successfully implemented a modular pipeline where each component is independently verifiable through unit testing and assertions[cite: 10, 63, 166]. The use of a standardized directory structure and a Makefile ensures that the entire analysis, from raw data to the final report, is fully reproducible with a single command[cite: 75, 80].

- **Predictive Utility:** The convergence of the learning curves and the close alignment in the temporal validation plots demonstrate that the system is a viable tool for traffic volume forecasting. The Mean Absolute Error (MAE) achieved (~ 70 vehicles) provides a high degree of confidence for engineering applications such as infrastructure planning and traffic light optimization.

- **Software Quality:** The integration of error handling, documentation (docstrings), and version control allowed for a robust collaborative environment[cite: 11, 166]. This rigorous approach minimizes the "technical debt" often found in scientific software, ensuring the long-term sustainability of the code.

In summary, the objective of building a cohesive, modular, and reproducible system has been met. The synergy between mathematical modeling and software engineering standards proved essential in transforming raw vehicular data into actionable scientific insights.

Table 1
Comparison of polynomial regression models.

Model	R^2 **train**	**RMSE train**	R^2 **CV**	**RMSE CV**	σ **RMSE CV**
Linear (degree 1)	0.2590	327.84	0.2284	328.81	52.36
Quadratic (degree 2)	0.5308	260.89	0.5061	262.74	42.67
Cubic (degree 3)	0.5771	247.67	0.5437	252.28	56.81

Table 2

Per-fold cross-validation metrics for the cubic model.

Fold	**RMSE**	R^2
1	319.66	0.4689
2	179.23	0.7284
3	267.18	0.5300
4	192.92	0.6287
5	302.42	0.3624
Mean	**252.28**	**0.5437**

Table 3

Input features used in C4 — all derived from the `date` *and* `hour` *columns of the processed dataset.*

Feature Pair	Source	Formula	Importance
`hour_sin`, `hour_cos`	`hour` (0–23)	$\sin / \cos\left(\frac{2\pi h}{24}\right)$	High
`hour_sin2`, `hour_cos2`	`hour` (2nd harmonic)	$\sin / \cos\left(\frac{4\pi h}{24}\right)$	Medium
`weekday_sin`, `weekday_cos`	`weekday_number` (0–4)	$\sin / \cos\left(\frac{2\pi w}{5}\right)$	Medium
`month_sin`, `month_cos`	`date.month` (1–12)	$\sin / \cos\left(\frac{2\pi m}{12}\right)$	High

Table 4

C4 — Model performance comparison on the test set (107 hourly samples, Feb 23 – Mar 03, 2026). All RMSE values are in vehicles/hour. Bold indicates best test performance.

Model	**$RMSE_{train}$**	**$RMSE_{test}$**	**R^2_{test}**	**CV-5 RMSE**
Linear Regression	97.06	38.02	0.276	185.84
Decision Tree	33.46	11.85	0.930	171.32
Random Forest	26.13	**10.99**	**0.939**	**147.97**

Table 5

Comparison of the RMSE (Root Mean Square Error) metric across the predictive models developed in the computational pipeline.

Predictive Model	Module	RMSE
Linear Regression	C2	327.800
Quadratic Regression	C2	260.900
Cubic Regression	C2	247.700
Optimized Linear Regression	C3	53.780
Linear Regression (Machine Learning)	C4	38.017
Decision Tree (Machine Learning)	C4	11.853
Random Forest (Machine Learning)	C4	10.994
Deep Learning Model (MLP)	C5	125.300

Table 6
Results after traffic flow simulation.

Métrica	Valor	Desviación
Tiempo promedio	15.4 s	1.2
Flujo vehicular	120 veh/h	5.0

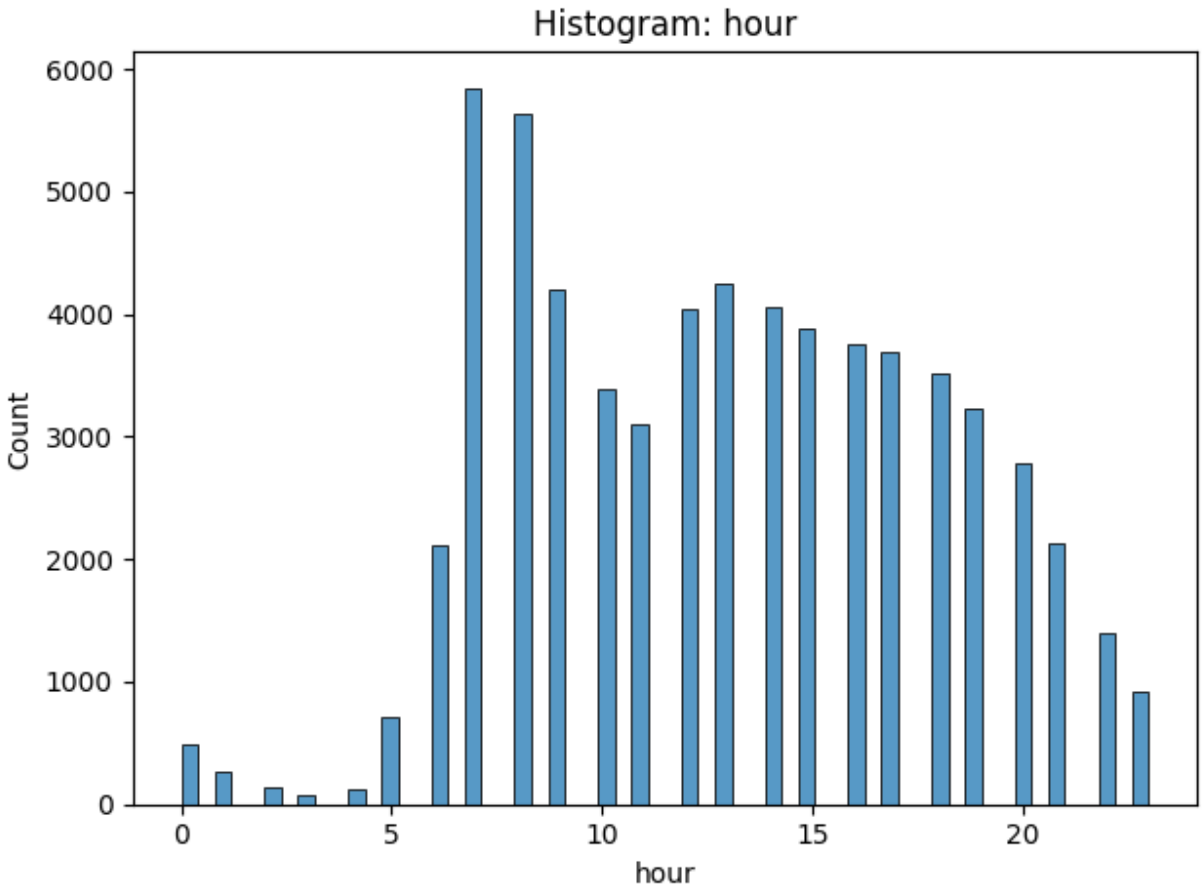

Figure 1
Histogram of dataframe entries by hour.

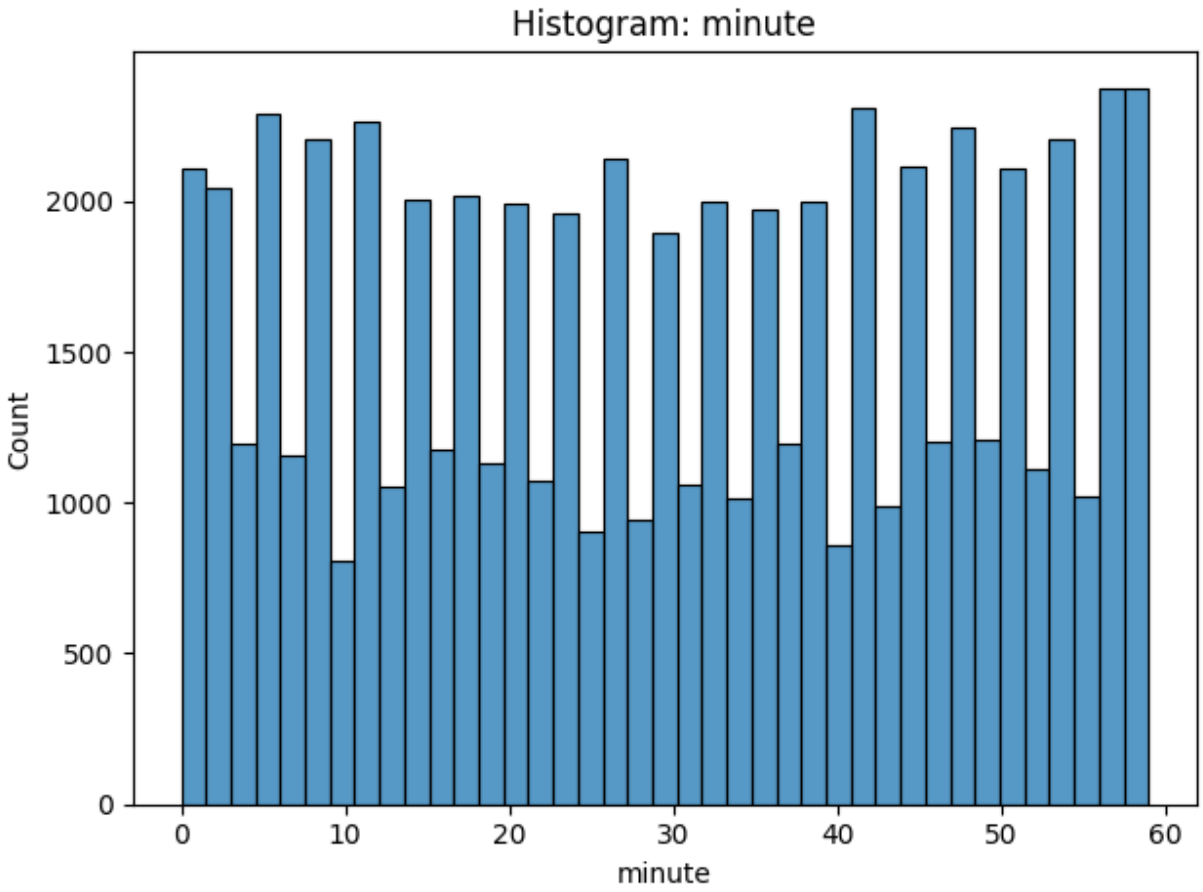

Figure 2
Histogram of dataframe entries by minute.

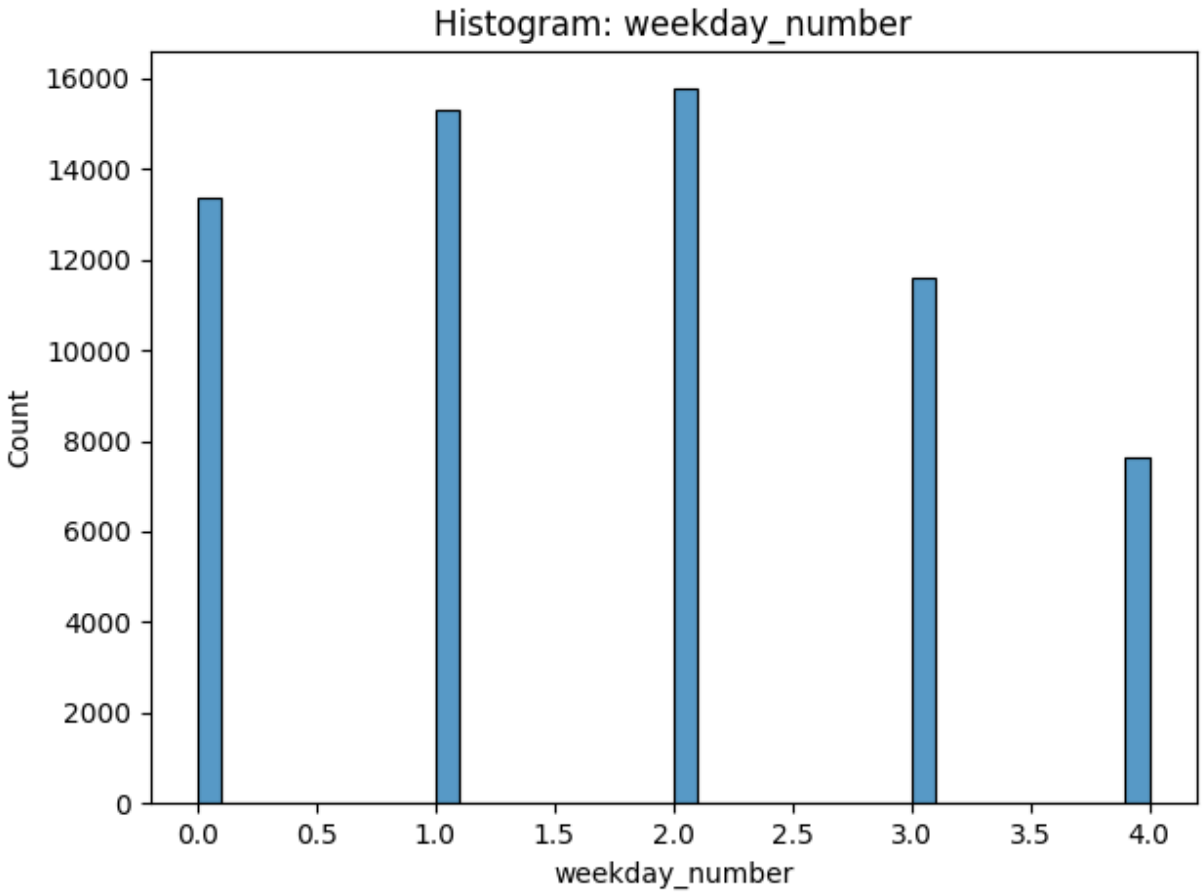

Figure 3
Histogram of dataframe entries by Weekday.

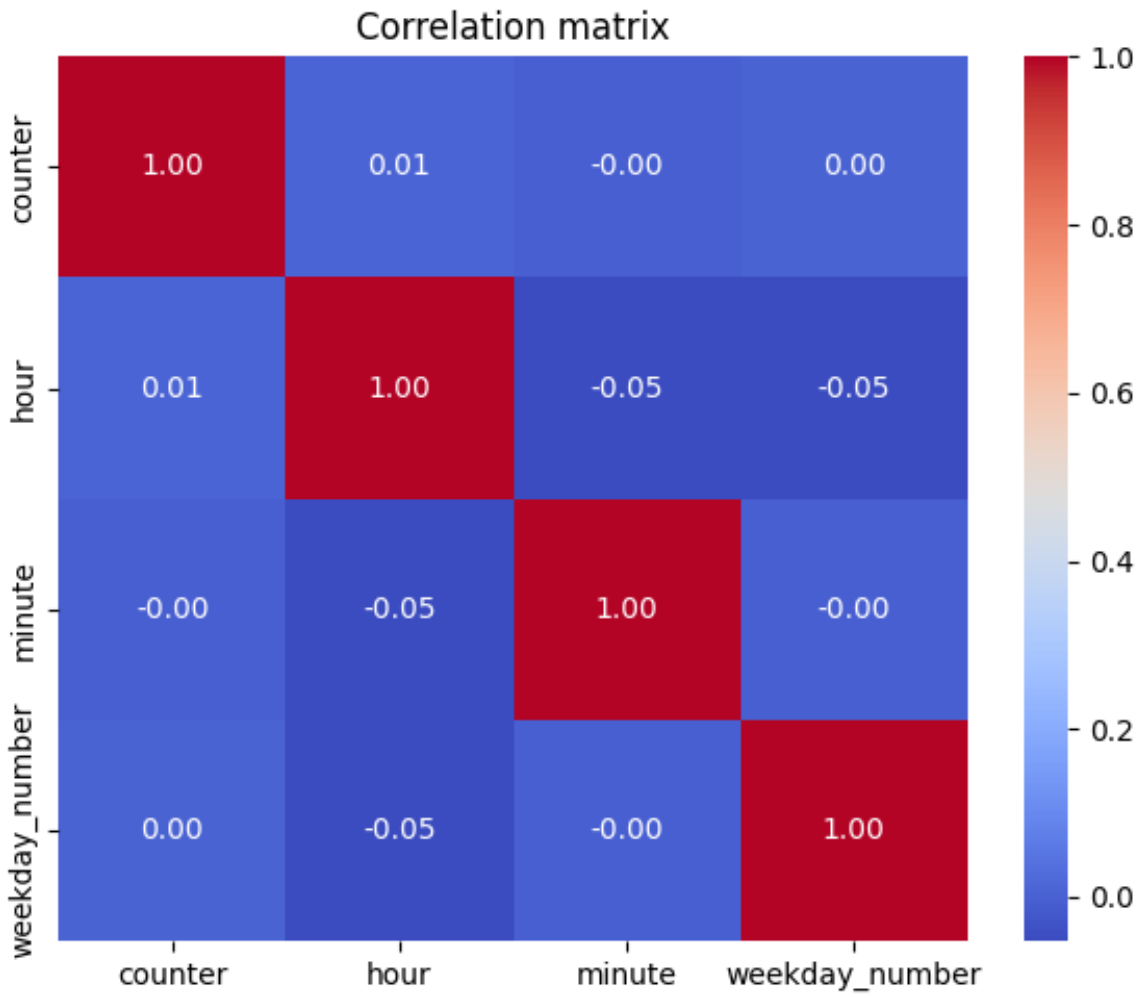

Figure 4
Correlation matrix of the dataframe columns.

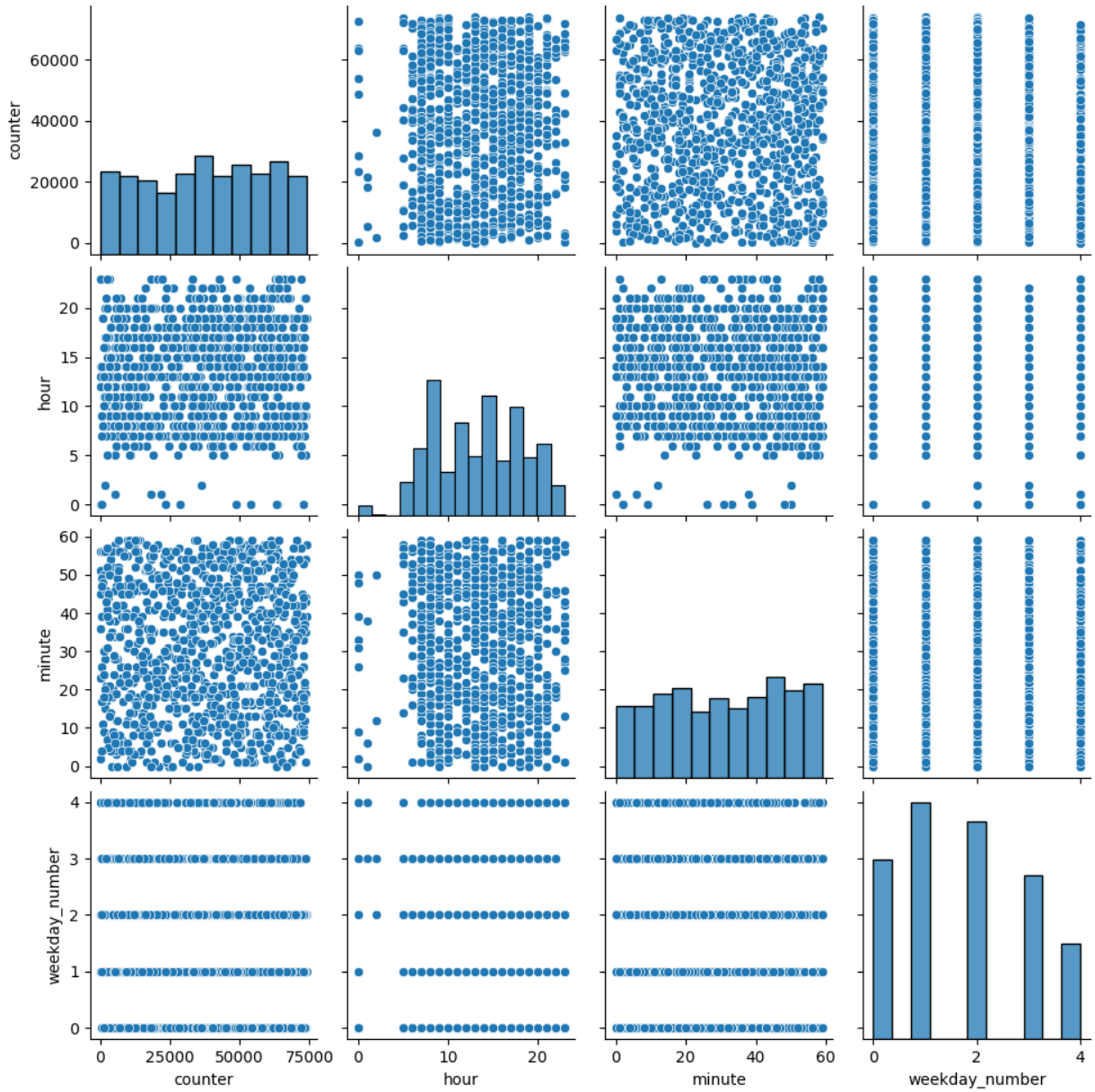

Figure 5

Pairplot matrix of the dataframe columns.

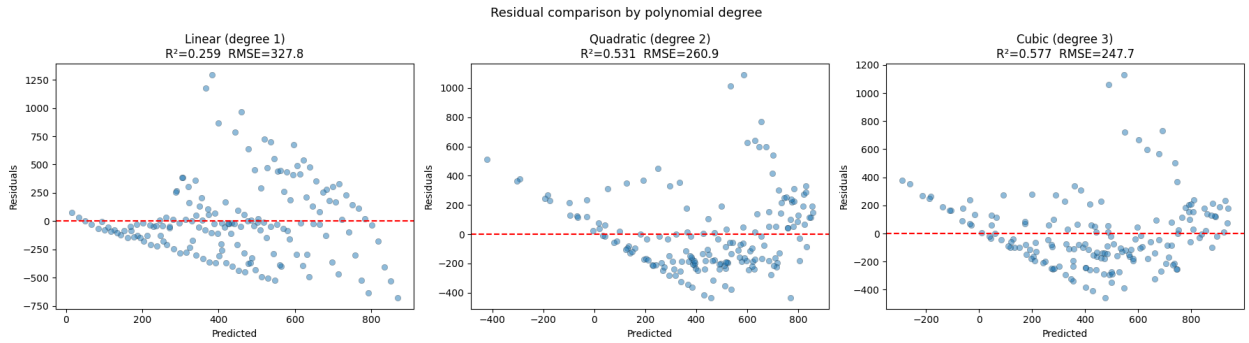

Figure 6

Residual comparison across polynomial degrees. All three models exhibit similar structural patterns in the residuals, indicating that increasing the degree does not resolve the underlying mismatch between the model and the data.

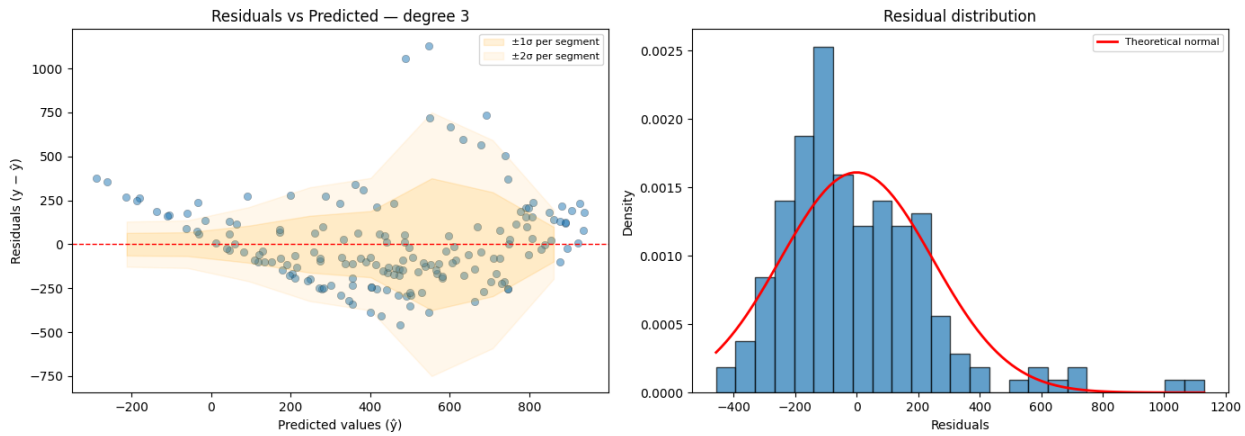

Figure 7

Diagnostic plots for the cubic model. Left: residuals versus predicted values with $\pm 1\sigma$ and $\pm 2\sigma$ bands per segment, showing clear heteroscedasticity. Right: histogram of residuals overlaid with the theoretical normal curve, showing positive skew and deviation from normality.

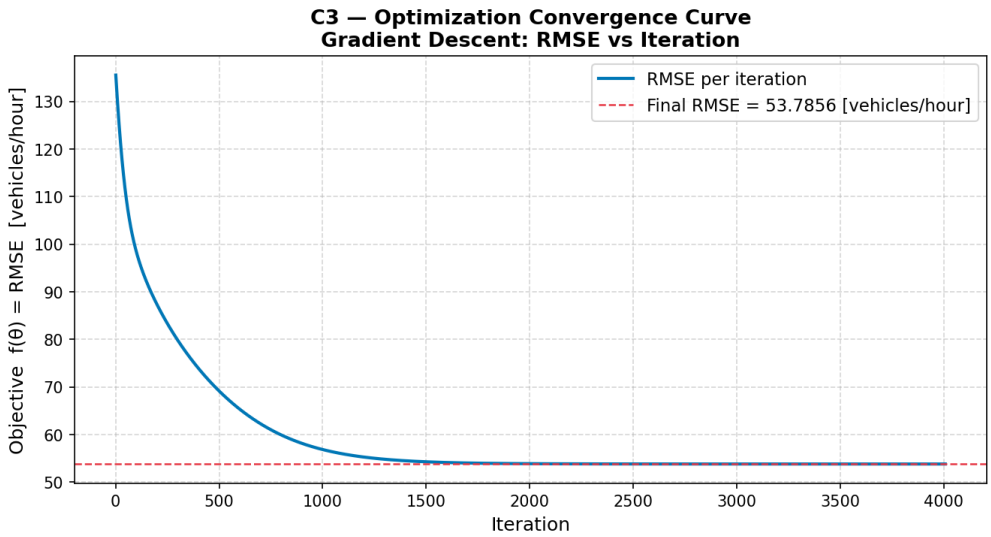

Figure 8

RMSE convergence curve during Gradient Descent optimization.

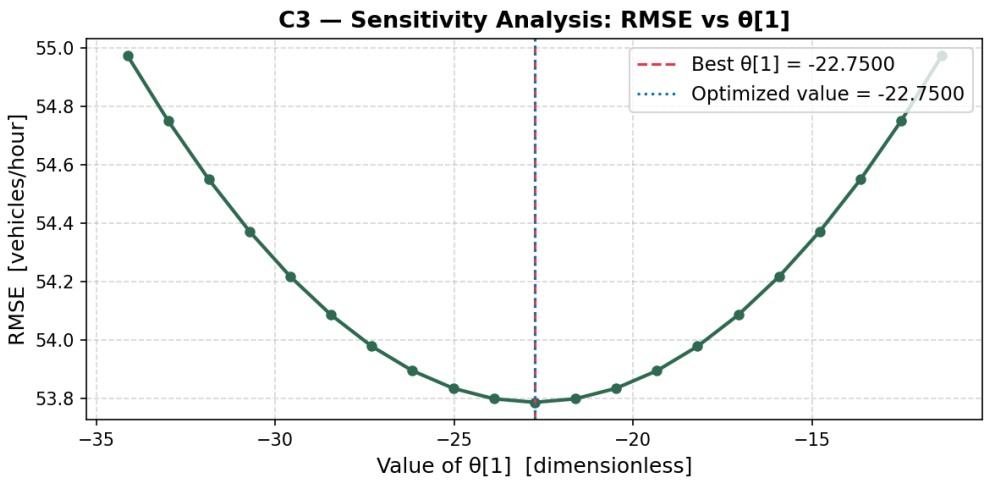

Figure 9

Sensitivity analysis of the RMSE with respect to parameter θ_1.

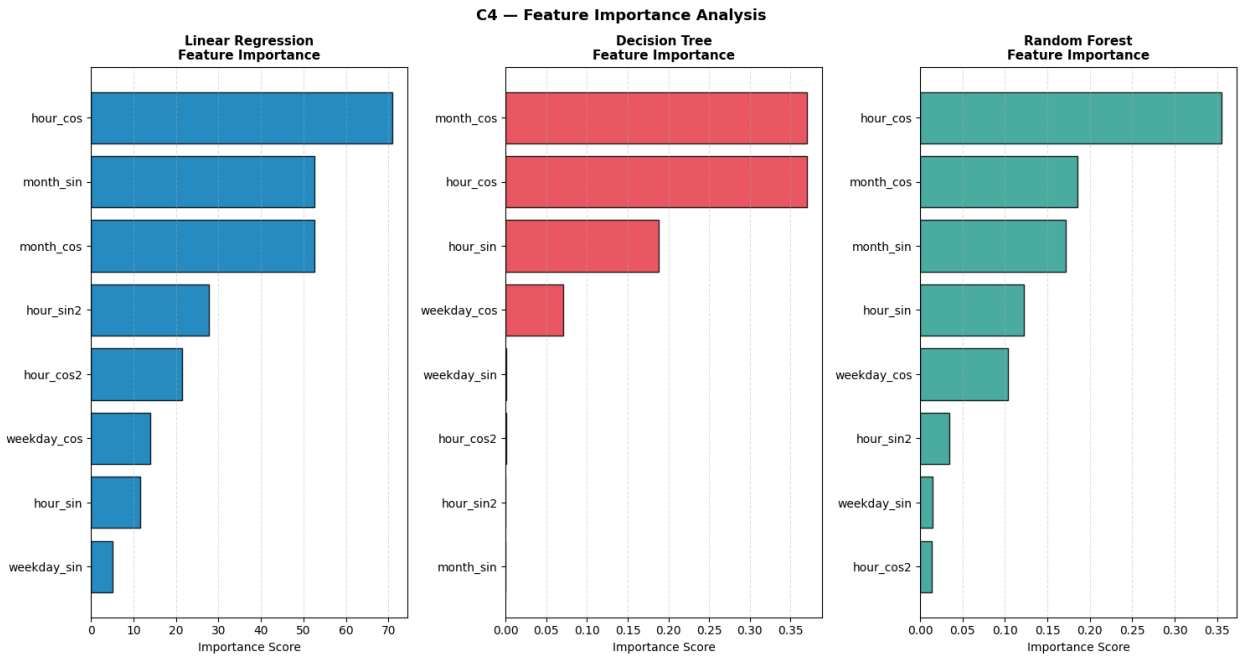

Figure 10

Feature importance scores for the Linear Regression (left) Decision Tree (center) and Random Forest (right). All models identify `month_cos` *and* `hour_cos` *as the dominant predictors of hourly traffic volume at the Nuevo Sur intersection.*

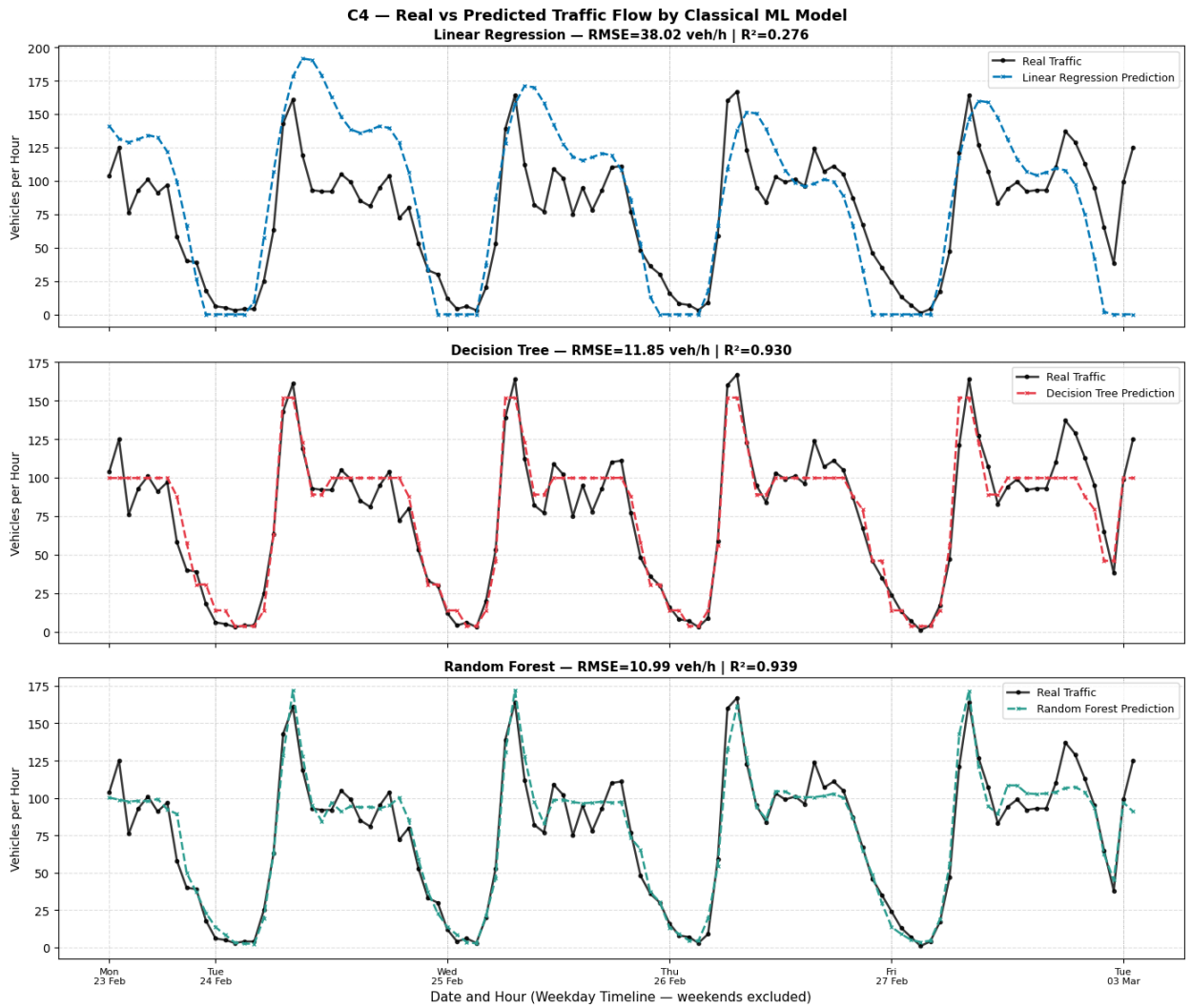

Figure 11

Actual vs. predicted hourly vehicle counts (vehicles/hour) during the test period (Feb 23 – Mar 03, 2026) for Linear Regression (top), Decision Tree (middle), and Random Forest (bottom). The X axis shows real dates and hours; weekends are excluded from the timeline.

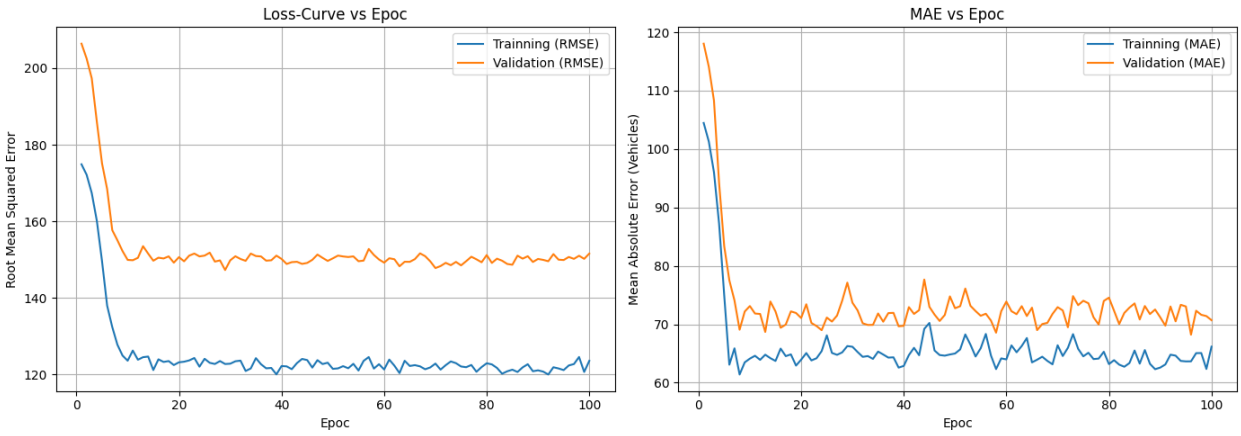

Figure 12

Learning curves: Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) per epoch.

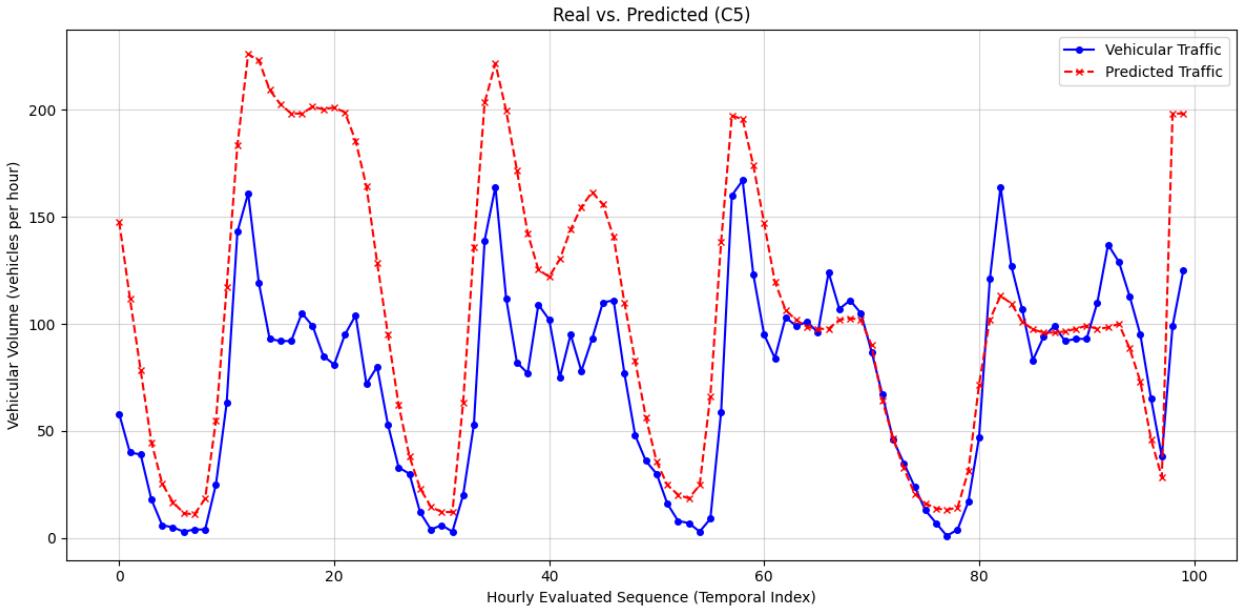

Figure 13

Temporal comparison between actual vehicular volume and MLP model predictions.